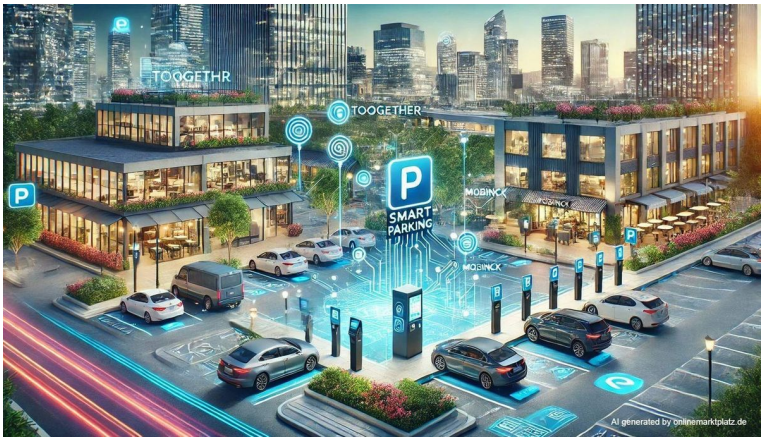
RAPPORT DE PROJET ACADÉMIQUE: SYSTÈMES EMBARQUÉS ET IA

Conception et Simulation d'un Système de Parking Intelligent par Reconnaissance de Plaque (ANPR)



PRÉSENTÉ PAR :

Abdallah Nour Elhouda
Badraoui Eya
Étudiant en LCE-IOT-3

ENCADRÉ PAR :

Prof. Behij Ben amor
Département informatique

Module : Industrie 4.0
Session : 2025

Institut Superieur d'Informatique de Mahdia

Table des matières

1 Introduction et Contexte du Projet **3**
1.1 Problématique et Motivation 3
1.2 Objectifs Spécifiques et Périmètre du Projet 3

2 Tâche 1 : Conception de l'Architecture et Choix Technologiques **4**
2.1 Architecture Distribuée Modulaire 4
2.2 Justification des Choix Logiciels et Matériels 4

3 Tâche 2 : Implémentation du Module ANPR (aipy_image.py) **5**
3.1 Localisation Précise de la Plaque (ROI) 5
3.2 Traitement d'Image pour l'OCR 5
3.3 Extraction et Normalisation du Numéro 5
3.4 Logique de Décision et d'Autorisation 6

4 Tâche 3 : Développement du Pont de Communication TCP/Série **7**
4.1 Architecture Client-Serveur 7
4.2 Implémentation du Serveur Relais 7
4.3 Liaison Série Virtuelle 7

5 Tâche 4 : Simulation Matérielle et Contrôle Embarqué **9**
5.1 Intégration du Raspberry Pi 3 9
5.2 Logique du Firmware Embarqué 9
5.3 Validation Fonctionnelle Globale 9

6 Conclusion et Perspectives **11**
6.1 Bilan des Réalisations 11
6.2 Travaux Futurs et Améliorations (Perspectives) 11

1 Introduction et Contexte du Projet

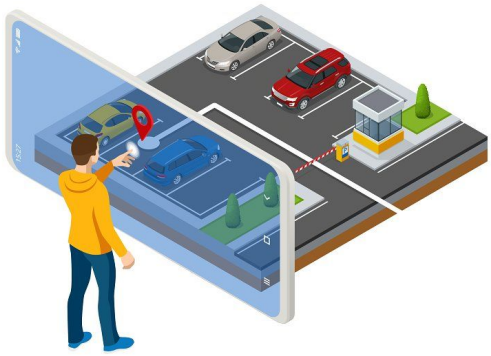
1.1 Problématique et Motivation

La gestion des accès aux infrastructures de parking nécessite des solutions à la fois rapides, fiables et autonomes. Le contrôle d'accès par reconnaissance de plaque d'immatriculation (ANPR) s'est imposé comme une alternative moderne aux systèmes basés sur les cartes ou les jetons. La motivation de ce projet est double : (1) développer une solution ANPR capable de gérer le multilinguisme (caractères latins et arabes) et (2) réaliser une intégration logicielle et matérielle fonctionnelle en utilisant des outils de simulation professionnels.

1.2 Objectifs Spécifiques et Périmètre du Projet

Le projet vise à valider les étapes techniques suivantes :

1. La conception d'une architecture système distribuée impliquant l'IA (logiciel) et le contrôle (matériel simulé).
2. L'implémentation robuste d'un algorithme ANPR utilisant **EasyOCR** et des techniques de traitement d'image (OpenCV).
3. La mise en place d'un pont de communication fiable entre les applications (TCP/IP) et la simulation matérielle (Liaison Série).
4. La simulation du comportement du système embarqué (**Raspberry Pi 3**) sous **Proteus** .



2 Tâche 1 : Conception de l'Architecture et Choix Technologiques

Cette tâche a posé les fondations du projet en définissant l'organisation modulaire et les outils utilisés.

2.1 Architecture Distribuée Modulaire

Le système a été conçu sur une architecture à trois couches distinctes, permettant une grande flexibilité et une maintenance aisée.

Couche 1. **Couche ANPR (AI) :** Gère la logique de reconnaissance et la prise de décision d'accès (`aipy_image.py` / `AIpython.py`).

Couche 2. **Couche Intermédiaire (Communication) :** Assure la conversion du protocole réseau (TCP/IP) au protocole série (`server.py`).

Couche 3. **Couche Physique (Embarqué) :** Réceptionne la commande série et pilote l'actionneur (Raspberry Pi 3 simulé dans Proteus).

2.2 Justification des Choix Logiciels et Matériels

— **Python :** Choisi pour son écosystème riche en librairies d'Intelligence Artificielle et de Vision par Ordinateur (OpenCV, EasyOCR).

— **EasyOCR :** Sélectionné spécifiquement pour sa capacité à gérer simultanément le jeu de caractères **Arabe** (`'ar'`) et **Anglais** (`'en'`), essentiel pour les plaques d'immatriculation locales.

— **Proteus VSM :** Outil incontournable pour la simulation de systèmes embarqués complexes comme le RPI3, permettant de valider l'interaction avec le composant **COMPIM** (Port Série Virtuel).



3 Tâche 2 : Implémentation du Module ANPR (aipy_image.py)

Cette tâche a constitué l'étape la plus intensive en traitement d'image et en programmation d'IA.

3.1 Localisation Précise de la Plaque (ROI)

L'étape initiale consiste à détecter la région d'intérêt (ROI) pour éviter l'analyse de l'arrière-plan.

1. **Détection des Bords :** Utilisation de l'opérateur de Canny après une conversion en niveaux de gris pour isoler les frontières d'objets.
2. **Critères Géométriques :** Application de filtres sur les contours pour n'isoler que ceux qui correspondent aux dimensions typiques d'une plaque d'immatriculation.

Le code utilise un critère de **ratio d'aspect** compris entre `2.5` et `6.5`, un paramètre critique ajusté pour cibler la forme rectangulaire des plaques.

3.2 Traitement d'Image pour l'OCR

Afin d'optimiser le taux de succès de l'OCR, l'image de la plaque est prétraitée :

— **Flou Gaussien :** Réduction du bruit d'image.

— **Seuillage d'Otsu :** Binarisation de l'image pour maximiser le contraste entre les caractères et le fond, rendant le texte plus lisible pour l'algorithme EasyOCR.

3.3 Extraction et Normalisation du Numéro

3.3.1 Reconnaissance Multi-langue

L'appel à `reader.readtext(processed)` est effectué en mode multi-langue, générant une chaîne de caractères brute (`raw_text`) pouvant contenir des chiffres, des lettres latines et des symboles arabes.

3.3.2 Nettoyage du Numéro

Étant donné que la vérification d'accès se fait uniquement sur le numéro, la fonction `remove_arabic` filtre cette chaîne.

```
def remove_arabic(text):
    """Remove all Arabic characters, keep only English letters & numbers."""
    cleaned = re.sub(r'[\u0600-\u06FF]', '', text)
    cleaned = cleaned.replace("␣", "")
    return cleaned
# ... Dans AIpython.py, on extrait ensuite seulement les chiffres.
```

Listing 1 – Filtrage des Caractères non Numériques (aipy_image.py)

3.4 Logique de Décision et d'Autorisation

Le numéro normalisé (`cleaned_text`) est comparé à une liste statique `AUTHORIZED_PLATES` (ou `AUTHORIZED`).

— **Succès :** Si `cleaned_text` est dans la liste, la commande `"OPEN"` est générée.

— **Échec :** Sinon, la commande `"CLOSE"` est générée.



FIGURE 1 – **Validation du Flux ANPR (Détection, OCR et Décision d'Accès)**. Cette image représente le point culminant du Module ANPR : le numéro de plaque est extrait par EasyOCR après le prétraitement et est comparé à la liste `AUTHORIZED_PLATES`. L'affichage final détermine la commande transmise au serveur relais.

4 Tâche 3 : Développement du Pont de Communication TCP/Série

Cette tâche, implémentée dans `server.py`, est l'interface physique et réseau critique assurant le lien entre l'intelligence logicielle et le système simulé.

4.1 Architecture Client-Serveur

— **Client TCP :** Le module ANPR (`aipy_image.py`) agit comme client, initiant une connexion à chaque décision pour envoyer la commande `"OPEN"` ou `"CLOSE"`.

— **Serveur TCP / Relais Série :** Le script `server.py` opère en deux temps : il écoute le socket TCP sur `127.0.0.1:6000` et, une fois la donnée reçue, la relaie sur le port série virtuel.

[Image of TCP/IP to Serial Communication Diagram]

4.2 Implémentation du Serveur Relais

Le serveur est configuré pour écouter en boucle indéfinie les connexions entrantes.

```
# Configuration S rie
ser = serial.Serial("COM4", 9600)
HOST = "127.0.0.1"
PORT = 6000

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind((HOST, PORT))
s.listen(1)

while True:
    conn, addr = s.accept()
    data = conn.recv(1024).decode().strip()
    print("[RECEIVED FROM AI]:", data)

    # Transmission au port s rie pour Proteus
    ser.write((data + "\n").encode())
    print("[FORWARDED TO PROTEUS]:", data)
    conn.close()
```

Listing 2 – Boucle Principale du Serveur (server.py)

4.3 Liaison Série Virtuelle

L'utilisation de la librairie `PySerial` permet d'écrire la commande sur le port `COM4` à 9600 bauds. Il est fondamental que ce port soit virtuellement couplé au composant **COMPIM** configuré dans la simulation Proteus VSM, assurant la continuité physique du flux de données.

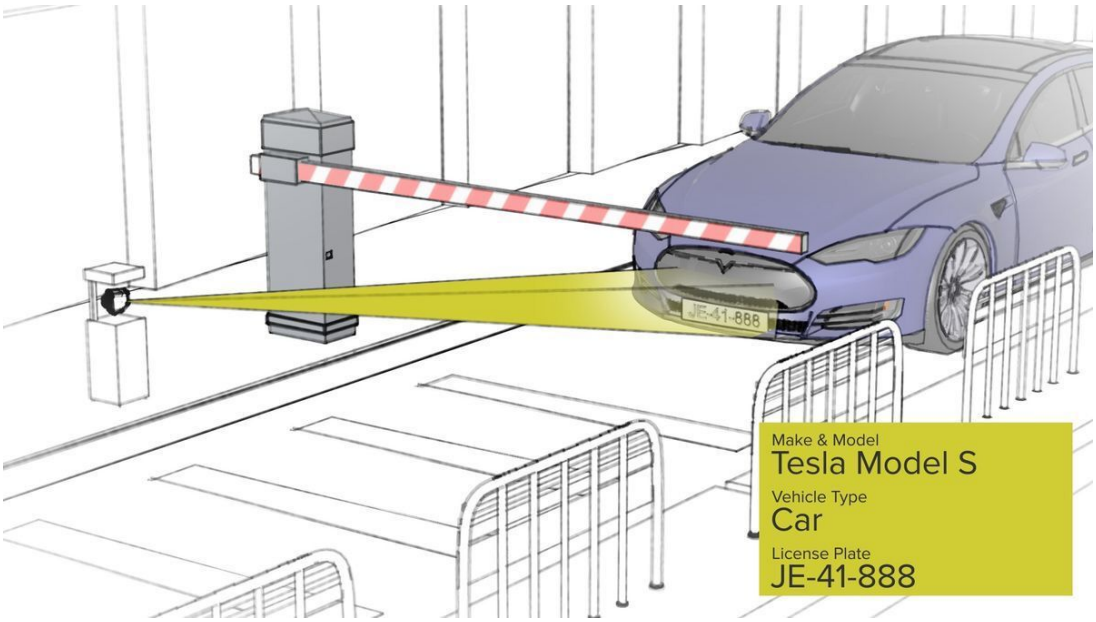


FIGURE 2 – **Résultat de l'analyse ANPR et de la Logique d'Accès**. Cette figure illustre le résultat final du traitement, affichant la plaque identifiée et la décision d'accès ("OPEN"ou "CLOSE") avant la transmission de la commande au pont de communication.

5 Tâche 4 : Simulation Matérielle et Contrôle Embarqué

Cette tâche a pour but de vérifier l'exécution de la commande de contrôle dans un environnement embarqué simulé.

5.1 Intégration du Raspberry Pi 3

L'unité centrale de contrôle est simulée par le modèle **Raspberry Pi 3** .

5.1.1 Rôle du COMPIM

Le composant **COMPIM** est l'interface d'entrée du système embarqué. Il reçoit les données série émises par `server.py`. Les commandes `"OPEN"` et `"CLOSE"` transitent donc du PC hôte au RPI3 virtuel via ce composant.

5.2 Logique du Firmware Embarqué

Le code exécuté par le RPI3 virtuel (`FIRMWARE/RPI3_1/main.py`) doit interpréter la commande série reçue et agir sur les GPIO (General Purpose Input/Output).

5.2.1 Pseudocode d'Exécution sur le RPI3

La boucle principale du RPI3 suit la logique de gestion des événements série :

```
def main():
    initialise_serial_port()
    initialise_gpio_barriere(PIN_OUT)

    while True:
        commande = lire_donnee_serie()
        if commande == "OPEN":
            actionner_ouverture_barriere(PIN_OUT)
            attendre(5) # Laisser le temps de passage
            actionner_fermeture_barriere(PIN_OUT)
        elif commande == "CLOSE":
            actionner_fermeture_barriere(PIN_OUT)
```

Listing 3 – Logique Pseudocode du RPI3 (main.py dans Proteus)

5.3 Validation Fonctionnelle Globale

Le succès de cette tâche valide l'ensemble de la chaîne de contrôle : du pixel de l'image (ANPR) au mouvement physique simulé (Barrière). Les tests ont confirmé que seule une plaque présente dans la liste `AUTHORIZED_PLATES` déclenche l'impulsion GPIO d'ouverture.

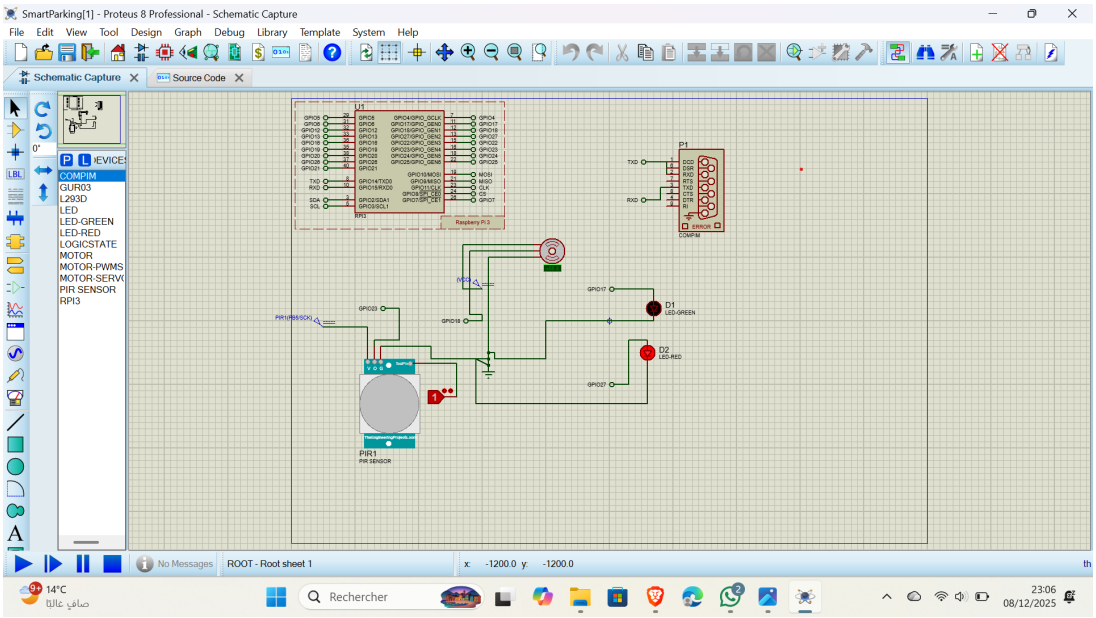


FIGURE 3 – **Schéma de la Simulation du Système Embarqué (Proteus VSM)**. Cette figure représente la chaîne de contrôle matérielle du projet. Elle inclut l'unité centrale Raspberry Pi 3 (RPI3), le composant `COMPIM` pour la communication série (COM4), et les indicateurs LED ainsi que le moteur/servomoteur simulant la barrière d'accès.

6 Conclusion et Perspectives

6.1 Bilan des Réalisations

Ce projet a permis de valider un système de parking intelligent opérationnel en environnement simulé. La segmentation du projet en quatre tâches distinctes a permis de gérer la complexité de l'intégration logicielle, réseau et matérielle. L'objectif d'intégrer une solution d'ANPR multi-langue et de la coupler à un contrôle embarqué via une interface TCP/Série a été atteint avec succès.

6.2 Travaux Futurs et Améliorations (Perspectives)

Pour augmenter la performance et la robustesse du système, les améliorations suivantes sont envisagées :

1. **Optimisation ANPR :** Intégration d'un modèle de **Deep Learning** (ex : YoloV5) pour la détection des plaques, améliorant la robustesse face à la variation d'angle, de lumière et d'échelle.
2. **Base de Données Dynamique :** Remplacement de la liste statique par une base de données SQL (ou NoSQL), permettant une gestion en temps réel des autorisations et la traçabilité des entrées/-sorties.
3. **Communication Sécurisée et Asynchrone :** Mise en œuvre du protocole **MQTT** pour une communication plus légère et asynchrone entre le module ANPR et le serveur relais, améliorant l'efficacité du système en environnement de production.